

UCS503 – Software Engineering
MediSynth

A Hospital Management System with a Privacy-Preserving
Synthetic Patient Data Engine (SynthetiX)

Submitted By:

1024030787 – Aryan Sharma

1024030792 – Brahmdeep Singh Khanuja

1024030793 – Keerat Singh Brar

Group – 3C55

BE Third Year , COE

Submitted To - Ms. Nisha Thakur



THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

Contents

Contents	2
Chapter 1: Introduction	3
1.1 Background and Context.....	3
1.1.1 Problem Statement.....	3
1.2 Project Objectives	4
Chapter 2: Methodology and Design	5
2.1 System Architecture	5
2.1.1 High-Level Design	5
2.1.2 Technical Stack.....	5
2.2 Implementation Details	6
2.2.1 Module 1: Core Functionality.....	6

Chapter 1: Introduction

1.1 Background and Context

Hospital software teams increasingly need realistic patient data to build, test, and demonstrate new systems before they can be deployed. This need extends beyond production hospitals to academic and research settings, where teams frequently have no legitimate way to access real patient records at all. Real patient data is protected health information under regulations such as HIPAA (United States) and India's Digital Personal Data Protection (DPDP) Act, and cannot legally or ethically be shared with developers, students, or third-party testers without extensive data-use agreements that are impractical within a single academic semester.

MediSynth addresses this gap by pairing a fully functional hospital management system (HMS) with Synthetix, a synthetic patient data engine. Rather than relying on real records or simple field-masking, Synthetix generates entirely new, artificial patient data that preserves the statistical patterns found in real healthcare data — age-diagnosis correlations, prescription frequencies, appointment volumes — without representing any real individual. This lets our team, and potentially others in a similar position, build and validate a realistic hospital system without ever touching protected data.

1.1.1 Problem Statement

Student and early-stage development teams building healthcare software face four related obstacles:

- **Legal restriction:** real hospital records fall under HIPAA and the DPDP Act; hospitals cannot hand them to student developers or third-party testers without extensive data-use agreements that are impractical within a single semester.
- **Re-identification risk:** simple anonymization (removing names or ID numbers) is frequently reversible — prior research has shown that a large majority of a population can be uniquely re-identified from just three seemingly harmless fields such as date of birth, gender, and postal code. This makes basic field-masking an unreliable substitute for genuine privacy protection.
- **Unrealistic test data:** teams that avoid real data often fall back on small, hand-typed mock records that do not reflect real clinical patterns, producing systems that are never properly stress-tested before deployment.
- **Fragmented development:** without a single, realistic, shared dataset, each part of a hospital system — backend, frontend, mobile, analytics — tends to invent its own inconsistent mock data, causing integration rework later in the project.

MediSynth's core engineering problem is therefore twofold: build a complete, role-based hospital management system, and build a data engine capable of seeding that system with statistically realistic, privacy-safe data from day one of development.

1.2 Project Objectives

1. Deliver a functional, role-based Hospital Management System covering patient registration, appointment booking, doctor consultation, medical records, prescriptions, lab test workflows, and billing, built and integrated across four coordinated lanes (backend, frontend, mobile, data engine) within the 10-week project timeline.
2. Design and implement Synthetix, a statistical synthetic-data generation engine that produces a patient dataset (target size: 10,000 records) with zero real patient records, validated by comparing key summary statistics of the generated data against reference public healthcare distributions before each release.
3. Establish a working CI/CD pipeline that automatically builds and tests all four lanes on every change and deploys to a staging environment, so that a working, seeded, role-based demo of the system is available by the mid-semester evaluation.

These objectives were chosen to be measurable at this stage of the project: Objective 1 and 3 are directly demonstrable in the current submission, while full validation of Objective 2's statistical fidelity target will be reported in the final evaluation once the complete synthetic dataset has been generated and benchmarked.

Chapter 2: Methodology and Design

2.1 System Architecture

2.1.1 High-Level Design

MediSynth is built across four coordinated lanes that share a common PostgreSQL schema:

- **Backend (Node.js/Express):** exposes REST APIs for authentication, patient and appointment management, medical records, prescriptions, and billing. Implements JSON Web Token (JWT) based authentication and role-based access control (RBAC) so that admin, doctor, lab technician, pharmacist, and patient roles each see only the actions relevant to them. All sensitive actions (record updates, prescription changes, dataset access) are written to an audit log.
- **Frontend (React):** the primary web interface used by hospital staff — admin dashboards, doctor consultation views, and pharmacy/lab workflows.
- **Mobile app (Flutter):** a lightweight cross-platform client aimed at patient-facing use — booking appointments, viewing prescriptions and reports, and making payments.
- **Data engine (Synthetix, Python):** decoupled from the backend so it can be run independently to (re)generate a synthetic dataset at any time. It is the only component that ever writes patient-like data into the system during development, since no real patient data is available to the team.

The database uses a 13-table relational schema with multi-tenant isolation, so that in principle multiple hospitals could use a single deployment without any cross-visibility of each other's data — a design choice made early to keep the schema realistic even though the current deployment target is a single institution.

2.1.2 Technical Stack

- **Framework/Language:** Java with SpringBoot (backend API); React (web frontend); Python (Synthetix data engine).
- **Database:** PostgreSQL, accessed via a parameterised query layer to prevent SQL injection.
- **Tools:** Git/GitHub for version control; GitHub Actions for CI/CD; Postman for API testing; draw.io for UML and data-flow design documentation.
- **Libraries:** React.js, Axios, and JavaScript for the frontend; Java and Spring Boot for the backend, with Spring Security, JWT, and BCrypt for authentication; and Python with pandas, NumPy, and SciPy for synthetic data generation.

2.2 Implementation Details

2.2.1 Module 1: Core Functionality

This module covers the functionality completed and demonstrable at the time of this submission.

Authentication and access control. All API routes are protected by JWT-based authentication issued at login. A role-based middleware layer checks each request against the caller's role (admin, doctor, lab technician, pharmacist, or patient) before allowing access, and passwords are stored using bcrypt hashing rather than in plain text.

Core relational schema. The 13-table PostgreSQL schema currently covers patients, staff (with role), appointments, medical records, prescriptions, and billing, with foreign-key relationships enforcing that, for example, a medical record cannot exist without an associated patient, and a prescription cannot exist without an associated medical record.

Patient and appointment workflows. CRUD APIs are implemented for patient registration and login, doctor search and selection, and appointment booking with conflict checking against existing bookings. A corresponding set of screens exists in the React frontend for staff-side booking management.

Doctor consultation and medical records. Doctors can view a patient's medical history and update it with consultation notes through a dedicated API and UI flow, forming the foundation that the lab-test and prescription workflows (planned for Module 2) will build on.

Synthetix v1: rule-based synthetic data generation. The first version of the Synthetix pipeline is implemented in Python as a CSV-to-CSV generator. It profiles publicly available healthcare statistics (age-band distributions, disease prevalence by age group) to define sampling distributions, then generates synthetic patient records from those distributions rather than from any real record. This first-pass dataset is used to seed the PostgreSQL database so that every lane of the project has a consistent, realistic dataset to build and test against, instead of each lane inventing its own inconsistent mock data.

Audit logging. Sensitive actions — record creation, updates to medical records or prescriptions — are written to an audit log table, laying the groundwork for the fuller data-governance features planned for later modules.

Together, these pieces form a working, seeded, role-based slice of the system: a user can register, log in under the correct role, book or manage an appointment, and view or update a medical record, all against data that Synthetix generated rather than any real patient's information.